ArXiv.org Dataset Classification

Interview task for Iris.ai

Shtiliyan Uzunov

Overview

The task proved to be very interesting, and a challenging one. I started with data exploration, and immediately noticed that the problem can be decomposed into two subproblems - parent classification, and subcategory classification. Using two step classification was the approach that I've chosen, and to solve it I trained several modules (LORA adapter + classification head) for a BERT based model (SPECTER 2) - 3 adapters for the parent classification, and 8 for each subcategory. To achieve such training I had to write a lot of logic for data selection, so that the training is fair (equal class representation), and at the same time feasible for my PC. The evaluation metrics that I've achieved are 0.84F1 score for the parent classifier, 0.68F1 average score for the 8 subcategory classifiers, and 0.62F1 score for the end to end classification. I spent about 30 hours of coding time, 20hours of train time (on my machine). The results are bound by the hardware that I have, but can be improved - more on that in the later sections. Finally I vibe coded a django Rest API using Cursor + Opus4.6. To setup the application read the README.md file in the repo, and download the resources zip at https://drive.google.com/file/d/1InWlBnPN0Sabuw2rsC2EpgpayhVoE7h-/view?usp=sharing

1. Dataset exploration

Some of the data in the dataset is more than 30+ years old, and the structure of the dataset itself has gone through some major changes throughout the years, most notably - 2007 the taxonomy was standardized, to what is now called “modern taxonomy”. This change included renaming some categories, and creating parent categories for others. I found the official taxonomy provided by arxiv.org at https://arxiv.org/category_taxonomy and I stick to this category taxonomy throughout the solution. In order to do it I implemented the following functionalities:

- Hierarchical representation of the categories with 8 parent, 160 child categories (Fig. 1)
- Mapping functionality for the old categories to their respective new categories. For example solv-int is remapped to nlin.SI, alg-geom -> math.AG, etc. Implementation in remap_to_modern_standard()

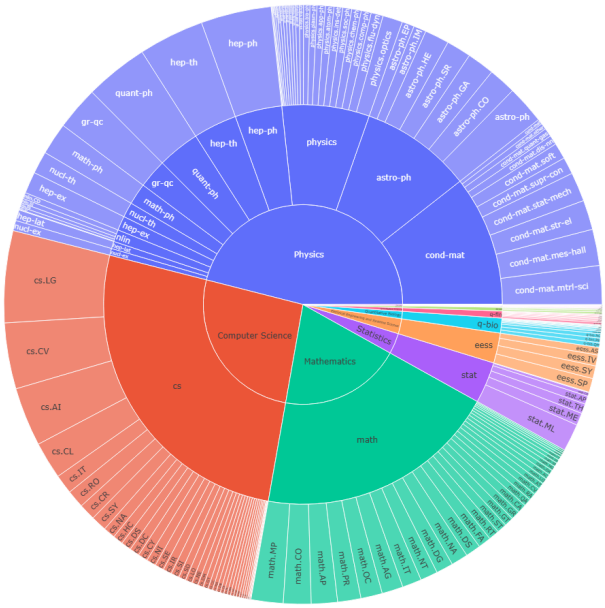

Fig. 1 Hierarchical representation of the categories. 3M documents ~5M category tags.

The legacy categories can be seen on Fig.2 They are 21 in total, not matching between the modern taxonomy, and what’s found in the dataset.

A good example of how categories have changed overtime can be seen in Fig. 3, that shows how the astro-ph category has been changed from a single big category, to a parent category, with several subcategories.

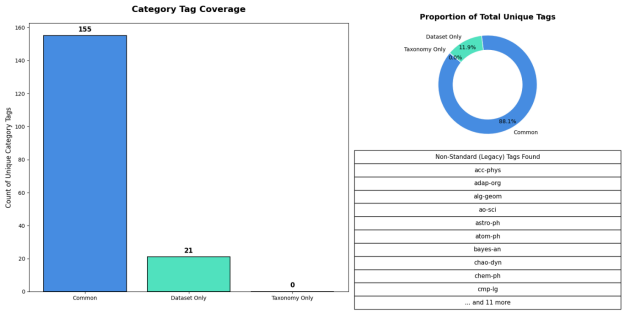

Fig. 2 Legacy (teal color) categories found in the dataset -. 21 total.

The ID field contains timestamp information for when the paper was published, I used them for date based filtering. The modern format is: YYMM.PublishOrder.

E.g. 2508.1234 is the 1234th paper published in August 2025.

The legacy IDs (pre 2007) were in format Subject/YYMMNumber.

E.g. hep-th/0602172 is paper published in high energy physics theory, published February 2006, number 172

Some general observations about the dataset are:

- Highly imbalanced representation of the categories. (as seen in Fig. 1)
- A paper can have multiple parent categories, with multiple child categories, making this a multilabel classification problem. Average 1.72 categories per paper.
- Multiple versions per paper - we have to always take the latest version of a paper during training.

Since the dataset is too big, to ease the work with it I implemented a parallel processor that parallelizes a given task to all CPUs available on the current machine. Implementation in /workers.

2. Classification strategy

I decided to split the problem in two - first I want to solve the parent classification problem, and then the subcategory classification. For example if the query abstract has a categories of [math.AC, cs.AI] the first classifier should predict parent categories [Mathematics, Computer Science], the second should predict [math->math.AC, cs->cs.AI]

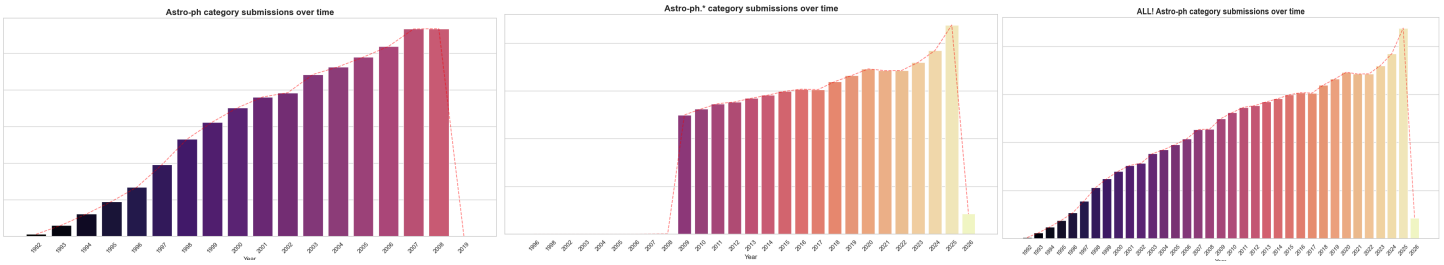

Fig. 3 Amount of papers with category astro-ph (1), astro-ph.* (2) and both combined. This illustrates the restructuring in 2007, when astro-ph turned into a parent category.

3. Model selection

There are two big directions we can choose from when solving the task - using LLMs (locally or through an API) without any training (my machine doesn't have resources) or using the classic discriminative approach (Encoder-only BERT like models). For this task I choose the latter one. When researching I found Specter2 - very suitable for the task as it already has been pretrained on similar data (abstract + title of scientific papers). This will be the model of choice for the task.

4. Data selection

To train the Specter2 I had to select balanced and fairly distributed data, so that all classes are equally represented - this prevents the model from biased predictions towards one specific class. On Fig. 4 we can see the distribution inside each category. I intentionally skipped the "Other/Legacy" category, as it seems to be a not well defined mix of different - thus we have 8 categories. The lowest represented category is Economics, with 14k samples. I decided to take 10k samples per class for the train dataset, and 2k samples per class for evaluation, so I ended up with ~80k train samples, and 16k test samples.

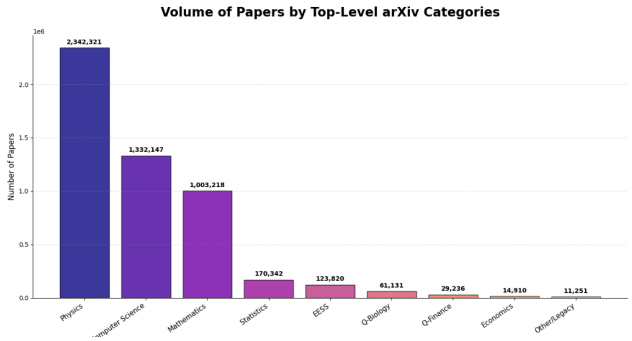

Fig.4 Volume of papers by Top-Level ArXiv categories.

After an initial evaluation, I got a score of 0.81F1, so I decided to add more data. To do this I implemented two additional adapters, and ensemble-like prediction, where I average the weights of the adapters. To train the the new adapters I created two new buckets, by sampling again 10k samples per class, with the restriction of not sampling from the test samples. Implementation can be found in the data_selection notebook. This strategy oversampled the Economics category, and added new samples from the other categories. The f1 changed from 0.81 to 0.84.

For the data selection of the subcategories I stuck with 1000 train samples per subcategory, and 200 test samples. Given the 160 unique subcategories this gave me a dataset of size 160k / 32k samples split among the different categories.

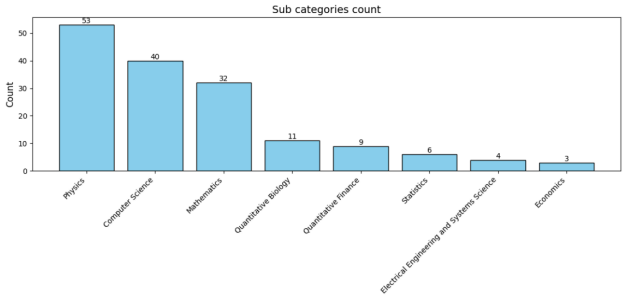

Fig. 5. Subcategories count

5. Tokenizer analysis

When working with BERT-like models it's important to analyze how much of the text can fit into the model, as these models have a fixed amount of tokens they can process (512 max). Luckily for this task the token distribution is fine, and I didn't have to implement any fancy strategies, out of 78k samples I had only 250 who were overflowing the max token count, so I decided not to implement anything for this case.

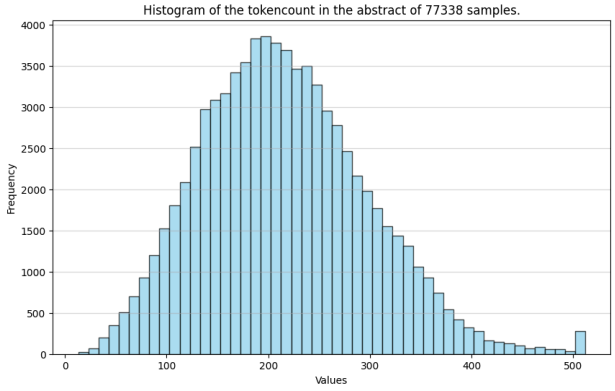

Fig. 6. Token count distribution

I also did analysis of [UNK] token counts. The tokenizer didn't face any unknown tokens, which means it splits all unknown tokens into subtokens. I analyzed the fertility rate, it has fr=1.31, which is not great, not terrible.

Based on these 3 metrics of the tokenizer I decided it's not needed to implement any type of data cleaning (e.g. special character removal, trimming). Being honestI was surprised by this, given that most texts are scientific, but I suppose that is because Specter2 is finetuned specifically on scientific papers and this behavior comes out of the box with the chosen model..

6. Training

My laptop has a NVIDIA GeForce RTX 4060 GPU (8GB RAM) which turned out to be suitable for the task. I intentionally made the dataset smaller (80k parent classification, 160k subclass classification). Trying to train on all 5Million samples would be very compute intensive, take much time, and I believe that the dataset size that I've chosen gave good results. Of course everything can be scaled (bigger dataset, more training epochs, etc.)

Adapter 1 in the parent classification was trained for 3 epochs, the other 2 for 1 epoch. As can be seen from the images - more epochs improve the F1, so that's a way we can get better results, if we have more time.

[11601/11601 3:41:02, Epoch 3/3]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.168900	0.175614	0.827542	0.825965
2	0.144400	0.165705	0.838012	0.836284
3	0.160200	0.162893	0.840713	0.839562

[3856/3856 1:41:13, Epoch 1/1]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.176600	0.169727	0.832456	0.831009

[3860/3860 2:18:44, Epoch 1/1]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.150400	0.169491	0.833190	0.832021

Each subcategory classifier was trained for 2 epochs. Here it's a lot more notable that more epochs give better score, but again - I didn't have time to wait for them all.

Physics:

[5214/5214 2:13:51, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.069800	0.065957	0.612225	0.583310
2	0.058300	0.062741	0.628857	0.604968

Maths:

[3154/3154 55:53, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.104400	0.098182	0.601576	0.599482
2	0.089700	0.090736	0.645325	0.644840

Computer Science:

[3834/3834 58:45, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.081000	0.077382	0.632250	0.612269
2	0.070100	0.073728	0.656113	0.637808

Q-Bio:

[1066/1066 6:11:55, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.183600	0.190244	0.614943	0.603729
2	0.167000	0.181761	0.641179	0.634825

Statistics:

[594/594 13:44, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.400400	0.390743	0.605528	0.582479
2	0.351700	0.377350	0.637046	0.615709

Q-Finance:

[828/828 17:33, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.272700	0.282321	0.543432	0.516723
2	0.256800	0.272363	0.584351	0.568707

Economics:

[300/300 06:28, Epoch 2/2]

Epoch	Training Loss	Validation Loss	F1 Micro	F1 Macro
1	0.280100	0.303288	0.817083	0.820246
2	0.230700	0.291073	0.826923	0.827729

As a conclusion - this can scale with more training time + more data.

7. Evaluation

The f1 scores of the 8 subcategory classifiers are seen in the images from the training section. Below are the results of the parent pipeline, tested with 1, 2 and 3 adapters.

```
Evaluation of the inference pipeline with 1 adapters
F1: 0.8324559267100062
Precision: 0.8075972373682297
Recall: 0.8588935709591371

Evaluation of the inference pipeline with 2 adapters
F1: 0.8398896304858496
Precision: 0.8187931661214104
Recall: 0.8621019595835885
```

```
Evaluation of the inference pipeline with 3 adapters
F1: 0.8395250433388634
Precision: 0.8185750636132315
Recall: 0.8615755442476183
```

Adding more adapters slightly improves the performance.

The f1 of the pipeline E2E (parent + subcat classifiers) is shown below:

```
Evaluation of the E2E pipeline. Evaluating on: 15893 samples.
F1: 0.6184050553112288
Precision: 0.5829583722443897
Recall: 0.6584414754203158
```

The pipeline has been tested locally, and no major issues were found.

8. REST API Service

Most of the service was vibecoded using Cursor + Opus4.6. You can check the instructions in the AGENTS.md file, and the description of the service in SPEC.md.

The service is written using the django framework as requested in the task and it implements basic logging functionality.

The API design is task + queue based -> when a request comes, the user gets a task ID, rather than an immediate result. This makes the system more robust in case of a big load. In production this should be implemented using a 3rd party system to preserve the tasks (for example Redis, or some database). I decided not to overcomplicate it, as adding a 3rd party will make the testing process harder, in case the reviewer tries to start the system.

Lastly, I implemented a semantic caching functionality. It calculates an embedding for the request, and tries to match it to already computed requests, the matching is done using cosine similarity, that's configurable through the .env file. The idea for the functionality I got from here . Unlike a traditional cache, where a cache hit happens only in a 100% match, here a cache might happen if the texts only partially match. For example if several words are changed - they will still hit the cache.

If you are going to test the system - don't forget to download the resources.zip file linked in the overview.